

Mamba-Based Graph Convolutional Networks: Tackling Over-smoothing with Selective State Space

Xin He¹, Yili Wang¹, Wenqi Fan², Xu Shen¹, Xin Juan¹, Rui Miao¹ and Xin Wang¹

¹Jilin University

²The Hong Kong Polytechnic University

{hexin20, wangyl21}@mails.jlu.edu.cn, wenqifan03@gmail.com, {shenxu23, junxin22, ruimiao20}@mails.jlu.edu.cn, xinwang@jlu.edu.cn,

Abstract

Graph Neural Networks (GNNs) have shown great success in various graph-based learning tasks. However, it often faces the issue of over-smoothing as the model depth increases, which causes all node representations to converge to a single value and become indistinguishable. This issue stems from the inherent limitations of GNNs, which struggle to distinguish the importance of information from different neighborhoods. In this paper, we introduce MbaGCN, a novel graph convolutional architecture that draws inspiration from the Mamba paradigm—originally designed for sequence modeling. MbaGCN presents a new backbone for GNNs, consisting of three key components: the **Message Aggregation Layer**, the **Selective State Space Transition Layer**, and the **Node State Prediction Layer**. These components work in tandem to adaptively aggregate neighborhood information, providing greater flexibility and scalability for deep GNN models. While MbaGCN may not consistently outperform all existing methods on each dataset, it provides a foundational framework that demonstrates the effective integration of the Mamba paradigm into graph representation learning. Through extensive experiments on benchmark datasets, we demonstrate that MbaGCN paves the way for future advancements in graph neural network research. Our code is in <https://github.com/hexin5515/MbaGCN>.

1 Introduction

In recent years, Graph Neural Networks (GNNs) [Zhang and Li, 2021] have gained significant attention for their ability to process graph data, achieving success in node classification [Shen *et al.*, 2024b; Shen *et al.*, 2024a], recommendation systems [Qin *et al.*, 2024], and biology [Shen *et al.*, 2025; Wang *et al.*, 2024c]. Among these, Graph Convolutional Networks (GCNs) [Wang *et al.*, 2024b; Wang *et al.*, 2022] stand out due to their capability to propagate node features through a graph’s topology and extract knowledge from non-Euclidean spaces (see Fig.1(a)). By using convolution operators, GCNs aggregate information from neighboring nodes,

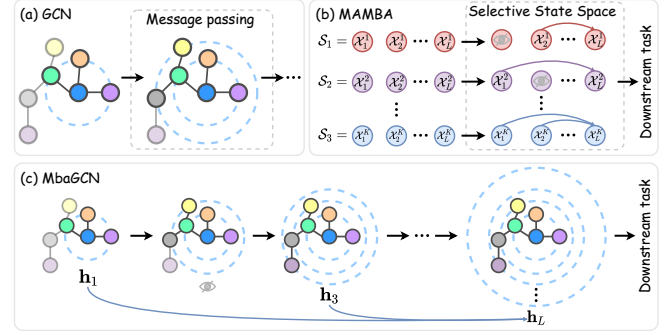


Figure 1: Comparison of GCN, MAMBA, and MbaGCN.

enabling effective learning for tasks such as link prediction and protein interaction analysis. However, GNNs face a key limitation: their struggle to effectively differentiate the significance of information coming from nodes located at different distances within the graph. This limitation directly leads to the issue of over-smoothing in GNNs. In other words, node representations gradually become more similar to each other as the depth of the network increases, limiting the scalability and performance of the GNNs.

Mamba [Gu and Dao, 2023; Dao and Gu, 2024], originally designed for sequence modeling, addresses a fundamental challenge in sequence-based tasks — the difficulty of capturing long-range dependencies. It incorporates a **selective state space mechanism** (see Fig.1(b)) that dynamically and adaptively compresses information from nodes at different distances, retaining only the most relevant data for the downstream tasks. This is crucial for tasks such as language modeling [Wang *et al.*, 2021; Yuan *et al.*, 2024] or time-series forecasting [Grazzi *et al.*, 2024; Wang *et al.*, 2025], where the importance of information diminishes with distance. The key insight of Mamba’s selective state space mechanism lies in its ability to differentiate the relevance of information at various distances [Zhu *et al.*, 2024]. Therefore, Mamba is inherently suited to address the over-smoothing problem in graph data, where nodes at different hops should contribute varying levels of importance during aggregation. Instead of uniformly aggregating information from all neighboring nodes, the Mamba-based approach adaptively aggregates based on each neighborhood’s relevance. This helps retain key fea-

tures from different-order neighborhoods, mitigating over-smoothing and improving the GCNs’ ability to capture multi-hop relationships (see Fig.1(c)).

In this work, we propose a novel architecture that integrates the Mamba paradigm into GNNs, named **Mamba-based Graph Convolutional Network (MbaGCN)**. Inspired by Mamba’s selective state space model, MbaGCN aims to address the limitations of traditional GCNs by adaptively compressing and propagating node features. Specifically, it preserves only the most relevant information for downstream tasks, enabling more effective aggregation and propagation.

MbaGCN consists of three key components: the **Message Aggregation Layer (MAL)**, the **Selective State Space Transition Layer (S3TL)**, and the **Node State Prediction Layer (NSPL)**. The MAL performs a simple message-passing operation that aggregates neighborhood information, helping nodes incorporate information from their neighbors. The S3TL introduces a selective state space mechanism that identifies and retains the most important neighborhood features, condensing the graph’s information into state vectors. This ensures that relevant node features are preserved, while redundant or less useful data is discarded. NSPL regulates the information flow within the same-order neighborhood, ensuring that essential local features are maintained while allowing the model to consider the global context. By alternating between these layers, MbaGCN balances local and global information propagation, adapting the information flow to the graph structure. The combination of MAL, S3TL and NSPL allows MbaGCN to scale effectively with deeper architectures and complex graph data, offering a promising solution to the challenges faced by traditional GCNs.

The contributions of this work are summarized as follows:

- We introduce a new approach to integrate the Mamba paradigm into GNNs, using its selective state space mechanism to address over-smoothing in graph representation learning.
- We propose MbaGCN, a new graph convolutional architecture that alternates between the MAL and the S3TL to adaptively retain important information from neighborhoods of different orders, while the NSPL refines the learned node representations.
- MbaGCN improves information flow through deeper GNN architectures by selectively retaining important features from neighborhoods, enabling the model to capture both local and global graph structures better.
- Our experiments on benchmark datasets demonstrate the potential of MbaGCN and provide valuable insights for future research directions in GNN development.

2 Related Work

2.1 Over-smoothing on Graph Data

Over-smoothing in graph representation learning arises as the number of GCN layers increases (repeated application of Laplacian smoothing) [Li *et al.*, 2018; Zhang *et al.*, 2021; Rusch *et al.*, 2023]. This leads to the convergence of all node representations within the same connected component of the graph to a single value, severely impacting the model’s

performance [Wu *et al.*, 2024; Zhai *et al.*, 2024]. In recent years, researchers have proposed various solutions to address this issue from different perspectives. For instance, introducing residual connections between layers preserves the integrity of node representations [Chen *et al.*, 2020; Zhu and Koniusz, 2021], employing personalized neighborhood aggregation based on PageRank better captures important node features [Chien *et al.*, 2020], and applying regularization techniques that incorporate both graph structure and node features mitigates over-smoothing [Yan *et al.*, 2022; Miao *et al.*, 2024]. These methods collectively enhance the model’s ability to retain meaningful node information across deeper layers and effectively mitigate the over-smoothing issue. Recent studies [Lieber *et al.*, 2024; Yang *et al.*, 2024; Hu *et al.*, 2025] show that Mamba can filter out irrelevant information from long sequential data, which inspires us to propose a Mamba-based GCN. This approach adaptively aggregates neighborhood information, mitigating over-smoothing and improving scalability in deeper models.

2.2 Mamba & Mamba with Graph

Mamba [Gu and Dao, 2023; Dao and Gu, 2024] was originally designed for sequence modeling that selectively filters and compresses information to retain only the most relevant data. Research on Mamba is still in its early stages, but some works have already focused on using Mamba to process graph data. Such as using Mamba in series with GCN improves the prediction of patients’ health status [Tang *et al.*, 2023], combining Mamba and GCN in parallel overcomes GCN’s limitation in capturing long-range dependencies between distant nodes [Wang *et al.*, 2024a; Behrouz and Hashemi, 2024], and applying Mamba directly captures long-distance dependencies between nodes in the graph [Ding *et al.*, 2024]. **These methods primarily combine the independent Mamba and GCN modules in various ways, yet they fail to fully harness Mamba’s capabilities in graph-structured data processing.** Therefore, in this paper, we propose a Mamba-based Graph Convolutional Network (MbaGCN) that integrates Mamba into the graph convolution process in a more cohesive manner, effectively addressing the over-smoothing problem in graph representation learning.

3 Preliminary and Background

3.1 Mamba

Mamba [Gu and Dao, 2023] is a class of linear time-varying systems that map an input sequence $x(t) \in \mathbb{R}^L$ to an output sequence $y(t) \in \mathbb{R}^L$, utilizing a latent state vector $h(t) \in \mathbb{R}^{N \times L}$, a state matrix $\mathbf{P} \in \mathbb{R}^{N \times N}$, an input matrix $\mathbf{Q} \in \mathbb{R}^{N \times 1}$, and an output matrix $\mathbf{R} \in \mathbb{R}^{1 \times N}$. The relationship between these components is given by the following equations¹:

$$\begin{aligned} h'(t) &= \mathbf{P}h(t) + \mathbf{Q}x(t), \\ y(t) &= \mathbf{R}h(t). \end{aligned} \tag{1}$$

¹In the graph domain, the matrix $\mathbf{A}$ has a special meaning, so we modify the notation from Mamba: $\mathbf{A} \rightarrow \mathbf{P}$, $\mathbf{B} \rightarrow \mathbf{Q}$, and $\mathbf{C} \rightarrow \mathbf{R}$.

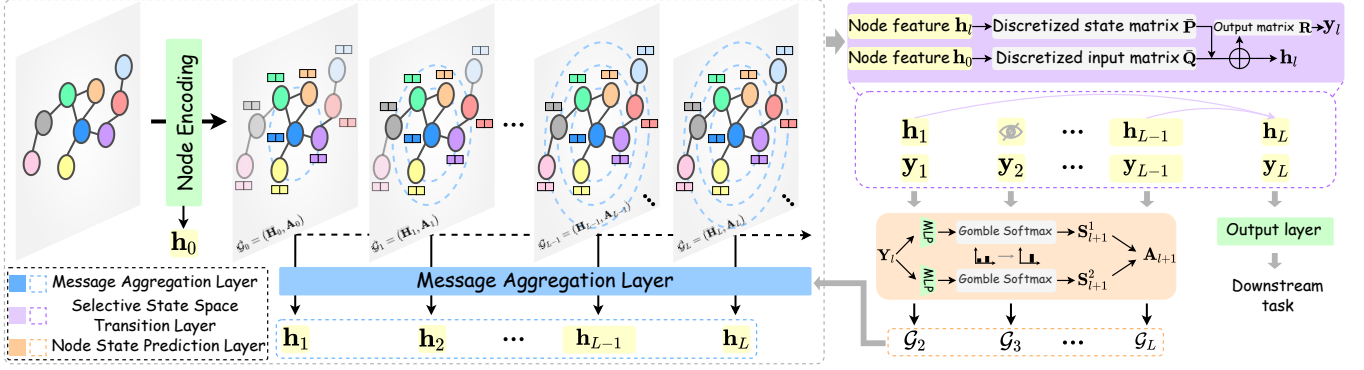


Figure 2: The Framework of MbaGCN.

Due to the challenges in solving the above equation within the deep learning paradigm, the discrete space state model [Gu *et al.*, 2021] introduces additional parameter Δ to discretize the aforementioned system, which can be formulated as follows:

$$\begin{aligned} h(t) &= \bar{\mathbf{P}}h(t) + \bar{\mathbf{Q}}x(t) \\ y(t) &= \mathbf{R}h(t) \end{aligned} \quad (2)$$

where

$$\begin{aligned} \bar{\mathbf{P}} &= \exp(\Delta \mathbf{P}) \\ \bar{\mathbf{Q}} &= (\Delta \mathbf{P})^{-1}(\exp(\Delta \mathbf{P}) - \mathbf{I}) \cdot \Delta \mathbf{Q} \end{aligned} \quad (3)$$

where $\bar{\mathbf{P}}$ and $\bar{\mathbf{Q}}$ are the discrete state matrix and discrete input matrix, $\exp(\cdot)$ refers to the exponential function with base e . On this foundation, Mamba further introduces a data-dependent state transition mechanism, which generates unique $\bar{\mathbf{P}}$, $\bar{\mathbf{Q}}$, $\mathbf{R}$, and Δ based on the input data, thereby achieving outstanding performance in language modeling. In this work, we adapt Mamba’s core ideas to construct a novel graph convolution paradigm that adaptively aggregates information from nodes across varying neighborhood orders. This adaptive aggregation helps address the issues of over-smoothing in deeper graph networks, which is a common challenge in traditional graph representation learning.

3.2 Problem Statement

Given an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with N nodes and M edges, where $\mathcal{V}$ is the set of N nodes and $\mathcal{E}$ is the set of M edges. We define the adjacency matrix as $\mathbf{A} \in \mathbb{R}^{N \times N}$, where $\mathbf{A}_{ij} = 1$ if there is an edge between node v_i and node v_j , and $\mathbf{A}_{ij} = 0$ otherwise. We also define the node feature matrix as $\mathbf{X} \in \mathbb{R}^{N \times d}$, which contains a d -dimensional feature vector for each node. In fully supervised node classification tasks, MbaGCN aims to optimize its parameters using the given training set of labeled samples and their corresponding ground truth $\mathbf{Y} \in \mathbb{R}^{N \times c}$ with the number of classes c , enabling the model to achieve better performance than shallow models even when the depth is increased to aggregate higher-order neighbors.

4 MbaGCN

The structure of MbaGCN, shown in Fig.2, includes the Message Aggregation Layer (MAL), Selective State Space

Transition Layers (S3TL), and Node State Prediction Layer (NSPL). The MAL aggregates neighborhood information using a basic graph aggregation operation. The S3TL fuses neighborhood features with node intrinsic features via a spatial state transition equation, condensing them into a state vector for the next iteration. MbaGCN alternates between MAL and S3TL, strategically modulating the influence of information from neighborhoods of varying orders within the node features. This approach effectively aligns the iterative Mamba paradigm with unordered graph structures, addressing the over-smoothing issue. Additionally, the NSPL is positioned between the MAL and S3TL, which regulates the information flow within the same-order neighborhood. This helps further refine the node feature representation.

4.1 Alternating MAL and S3TL

Graph representation learning [Xu *et al.*, 2021] relies on effectively aggregating neighborhood information to enhance node representations. However, traditional methods [Wan *et al.*, 2021; Li *et al.*, 2021] often face challenges such as over-smoothing, where node-specific features become indistinguishable as the network deepens. To address this, we introduce an alternating mechanism between the MAL and the S3TL. This alternating process refines the aggregation of neighborhood information, with each layer alternating between capturing local feature details and adaptively compressing relevant neighborhood information.

Message Aggregation Layers (MAL)

The MAL is the initial stage of MbaGCN, responsible for capturing local feature information by aggregating neighboring node features. It serves as the foundation for the feature refinement process, enabling effective aggregation of node information from the graph’s local structure. The MAL is computed as follows:

$$\mathbf{H}_l = \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{A} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}_{l-1}, \quad (4)$$

where $\tilde{\mathbf{D}}$ is the diagonal degree matrix, and $\mathbf{A}$ is the adjacency matrix. $\mathbf{H}_{l-1}$ and $\mathbf{H}_l$ represent the feature matrices of node after aggregating $(l-1)$ -hop and l -hop neighborhoods, respectively. The MAL efficiently aggregates the features of a node’s neighbors, yet it struggles to scale with deeper networks or more complex graph structures. This necessitates

the introduction of the S3TL, which utilizes the selective state space model to address this issue.

Selective State Space Transition Layers (S3TL)

Traditional neighborhood aggregation methods [Jin *et al.*, 2021; Yu *et al.*, 2022] often struggle to capture the full range of important information, especially when processing nodes at varying distances. To effectively aggregate information from diverse neighborhood orders without losing relevant features, S3TL adaptively combines neighborhood information with node-specific features. This process leverages a selective state space transition mechanism that compresses the aggregated data, retaining only the most pertinent information. At the same time, redundant information is discarded, improving the model’s adaptability to deeper layers and more complex graph structures.

To effectively handle the diverse structures and features of nodes, it is crucial that the fusion process in S3TL can adapt accordingly. To achieve this, we introduce the input-related approach [Gu and Dao, 2023] for generating the input matrix $\mathbf{Q}$, output matrix $\mathbf{R}$, and additional parameter matrix $\mathbf{\Delta}$, which are essential for the selective state space transition. These matrices are computed based on the initial feature matrix $\mathbf{H}_0$ of the nodes as follows:

$$\mathbf{Q} = \mathbf{H}_0 \mathbf{W}_Q, \mathbf{R} = \mathbf{H}_0 \mathbf{W}_R, \mathbf{\Delta} = \mathbf{H}_0 \mathbf{W}_\Delta, \quad (5)$$

where $\mathbf{W}_Q$, $\mathbf{W}_R$, and $\mathbf{W}_\Delta$ are learnable parameters in the S3TL. This adaptive matrix generation enables the model to derive distinct $\bar{\mathbf{P}}$, $\bar{\mathbf{Q}}$, and $\mathbf{R}$ for each target node, allowing the model to adaptively weight the node’s features and its neighborhood information, thus enhancing its adaptability. The matrices $\bar{\mathbf{P}}$ and $\bar{\mathbf{Q}}$ are discretized as follows:

$$\begin{aligned} \bar{\mathbf{P}} &= \exp(\mathbf{\Delta P}), \\ \bar{\mathbf{Q}} &= (\mathbf{\Delta P})^{-1}(\exp(\mathbf{\Delta P}) - \mathbf{I}) \cdot \mathbf{\Delta Q}, \end{aligned} \quad (6)$$

where $\exp(\cdot)$ refers to the exponential function with base e .

It is worth noting that the state matrix $\mathbf{P}$ needs to be initialized in a special way named HiPPO-LegS [Gu *et al.*, 2020], which can enable $\mathbf{P}$ to select useful information for downstream tasks. The initialization process is defined as follows:

$$\mathbf{P}[n, k] = - \begin{cases} (2n + 1)^{1/2}(2k + 1)^{1/2}, & \text{if } n > k \\ n + 1, & \text{if } n = k \\ 0, & \text{if } n < k \end{cases} \quad (7)$$

where n and k are the indices along the two dimensions of the state matrix $\mathbf{P}$.

After alternating between the MAL and S3TL, the final node representation $\mathbf{Y}_l$ is derived by applying the output matrix $\mathbf{R}$ to the aggregated features from the target node’s l -order neighborhood:

$$\begin{aligned} \mathbf{H}_l &= \bar{\mathbf{P}} \cdot \mathbf{H}_l + \bar{\mathbf{Q}} \cdot \mathbf{H}_0, \\ \mathbf{Y}_l &= \mathbf{R} \cdot \mathbf{H}_l, \end{aligned} \quad (8)$$

where $\mathbf{Y}_l$ is the representation of the nodes after aggregating the information from its l -order neighborhood. $\mathbf{H}_l$ contains the compressed information from the $(l - 1)$ hop neighborhood, which is adaptively refined through the discretized state

matrix $\bar{\mathbf{P}}$ and the input matrix $\bar{\mathbf{Q}}$ to better adapt to downstream tasks. This process enables the model to adaptively preserve important information from higher-order neighborhoods, discard redundant data, and retain the intrinsic features of the nodes. Through this iterative process, the node features are refined and enriched, leading to more accurate node representations.

4.2 Node State Prediction Layer (NSPL)

While the MAL and S3TL effectively refine neighborhood aggregation, they are limited in distinguishing the significance of different nodes within the same neighborhood. This limitation becomes particularly critical when the model aggregates higher-order neighborhood information, as the number of nodes to be aggregated grows exponentially with the model depth, significantly increasing the presence of redundant nodes within the neighborhood. To tackle this, we introduce the Node State Prediction Layer (NSPL). The main purpose of NSPL is further to regulate the information flow within the same-order neighborhood, allowing the model to prioritize the most relevant features and discard less important ones. This additional layer helps prevent the loss of key node-specific characteristics during message passing, ensuring that the model retains high-quality node representations even as it processes deeper graph layers.

In NSPL, we employ two parameter matrices $\mathbf{W}_1$ and $\mathbf{W}_2$ to predict the state of the target node while aggregating information from its neighborhood at various orders. Specifically, the state vectors $\mathbf{S}_l^1$ and $\mathbf{S}_l^2$ are derived by applying the Gumbel-Softmax function to the node representations after neighborhood aggregation. This function allows for differentiable discrete sampling, which is essential for gradient-based optimization in deep learning. The state vectors $\mathbf{S}_l^1$ and $\mathbf{S}_l^2$ determine which neighborhood nodes contribute to the aggregation process, effectively controlling how much influence the information from each neighbor should have. The equations for generating the state vectors are as follows:

$$\begin{aligned} \mathbf{S}_l^1 &= \text{Gumbel} - \text{Softmax}(\mathbf{Y}_{l-1} \mathbf{W}_1, \tau), \\ \mathbf{S}_l^2 &= \text{Gumbel} - \text{Softmax}(\mathbf{Y}_{l-1} \mathbf{W}_2, \tau), \end{aligned} \quad (9)$$

where $\mathbf{S}_l^1$ and $\mathbf{S}_l^2$ represent the predicted information flow when the target node aggregates data from its l -order neighborhood. The Gumbel-Softmax function is used for differentiable discrete sampling, enabling the model to optimize the information flow using gradient-based methods, even though the decision itself is discrete. The temperature parameter τ in the Gumbel-Softmax function controls the sharpness of the distribution. A lower value of τ results in more discrete outputs (closer to a one-hot vector), while a higher value introduces more randomness, promoting exploration during training. This mechanism allows for more flexible control of the information flow for each node during aggregation, enabling its contribution to be dynamically adjusted based on the scope of neighborhood aggregation.

To effectively manage the information flow between nodes and improve the model’s adaptability, we introduce a crucial adjustment step based on the state vectors $\mathbf{S}_l^1$ and $\mathbf{S}_l^2$. These state vectors are used to modify the adjacency matrix $\mathbf{A}_l$ during subsequent message aggregation, enabling the model to

control the information flow between different neighborhoods adaptively. This allows the model to fine-tune which neighbors' features should be aggregated, ensuring that only the most relevant information contributes to the node's updated representation. The modification of the adjacency matrix is formulated as follows:

$$\mathbf{A}_l[i, j] = \begin{cases} 1, & \text{if } \mathbf{A}[i, j] = 1 \wedge \mathbf{S}_{l,j}^1 = 1 \wedge \mathbf{S}_{l,i}^2 = 1 \\ 0, & \text{else} \end{cases} \quad (10)$$

In this formula, $\mathbf{A}_l[i, j]$ determines whether there is a information flow between nodes i and j when aggregating the l -order neighborhood information. The decision is made based on the state vectors $\mathbf{S}_{l,j}^1$ and $\mathbf{S}_{l,i}^2$, which are learned from previous neighborhood aggregations. By regulating the information flow in this manner, the model ensures that only the most relevant features from each node's neighborhood are propagated, while irrelevant or redundant features are filtered out.

This dynamic adjustment process empowers the NSPL to adaptively control the information flow at each layer during aggregation. As a result, the model becomes more efficient in propagating meaningful features while avoiding the influence of noise or less important information. This targeted aggregation process improves the overall effectiveness of the graph representation learning, ensuring that each node's updated representation is both accurate and informative.

4.3 Total Complexity of MbaGCN

The total time complexity of MbaGCN combines the complexities of MAL, S3TL, and NSPL. Since the alternating stacking of MAL and S3TL forms the core of the model, and the NSPL is applied after each layer, the total complexity for each layer is as follows: MAL and S3TL together contribute a time complexity of $\mathcal{O}(|\mathcal{E}|d + Nd^2)$ per layer, while NSPL adds an additional $\mathcal{O}(Nd^2)$ per layer. Assuming the model has L layers, the total time complexity of MbaGCN is $\mathcal{O}(L \cdot |\mathcal{E}|d + L \cdot Nd^2)$. In practice, when the number of nodes and feature dimensions is large, the $L \cdot Nd^2$ term typically dominates. Thus, the overall complexity remains primarily influenced by the graph size and feature dimensions across all layers. For a detailed implementation of MbaGCN, please refer to Algorithm 1.

5 Experiment

In this section, we conduct a series of experiments to evaluate MbaGCN's performance, comparing it with several widely used GNN architectures. The aim is to assess MbaGCN's effectiveness as a new backbone for graph representation learning, focusing on its ability to address key challenges such as over-smoothing and adaptability to various graph structures. All experiments are performed on a system with an Intel(R) Xeon(R) Gold 5120 CPU and an NVIDIA L40 48G GPU.

5.1 Experimental Settings

Datasets: We evaluate our method on a variety of datasets across different domains, focusing on full-supervised node classification tasks. The datasets include three citation graph datasets (**Cora**, **Citeseer**, **Pubmed**), two web graph datasets (**Computers**, **Photo**), and two heterogeneous graph datasets

Algorithm 1 Mamba-based Graph Convolution Network

Input: Adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, state matrix $\mathbf{P}$, learnable parameters $\mathbf{W}_Q$, $\mathbf{W}_R$, $\mathbf{W}_\Delta$, $\mathbf{W}_1$, $\mathbf{W}_2$.

Output: The updated node representations $\mathbf{Y}$.

```

1: Compute  $\mathbf{Q}$ ,  $\mathbf{R}$  and  $\Delta$  via Eq.5;
2: while not convergent do
3:   for  $l = 1 \rightarrow L$  do
4:     Compute  $\hat{\mathbf{H}}_l$  via Eq.4;
5:     Compute  $\mathbf{H}_l$  and  $\mathbf{Y}_l$  via Eq.8;
6:     Compute  $\mathbf{S}_l^1$  and  $\mathbf{S}_l^2$  via Eq.9;
7:     Modify the adjacency matrix  $\mathbf{A}_l$  via Eq.10;
8:   end for
9:   Obtain node representations  $\mathbf{Y}$ ;
10:  Update all learnable parameters via back propagation;
11: end while
12: return Updated node representations  $\mathbf{Y}$ .
```

(**Actor**, **Wisconsin**). For citation and heterogeneous graph datasets, we use the feature vectors, class labels, and 10 random splits as proposed in [Chen *et al.*, 2020]. For the web graph datasets, the same components are used following the protocol in [He *et al.*, 2021]. Detailed statistics and descriptions of these datasets can be found in Appendix A.1.

Baselines: To evaluate the effectiveness of MbaGCN, we compare it with several representative GNN models, including classical models like **GCN** [Kipf and Welling, 2016], **GAT** [Veličković *et al.*, 2017], and **SGC** [Wu *et al.*, 2019], as well as deep GNN models such as **APPNP** [Gasteiger *et al.*, 2018], **GCNII** [Chen *et al.*, 2020], **GPRGNN** [Chien *et al.*, 2020], **SSGC** [Zhu and Koniusz, 2021], and **GGCN** [Yan *et al.*, 2022]. Further details about these baseline models can be found in Appendix A.2.

5.2 Performance Evaluation of MbaGCN

Q: Does MbaGCN outperform baseline models across various datasets? Yes, MbaGCN consistently achieves the highest average rank across all datasets, demonstrating its overall adaptability and robustness.

▷ **Performance across Diverse Datasets:** As shown in Tab.1, MbaGCN consistently outperforms all baseline models, achieving an average rank of 1.71, and ranks first on six out of eight datasets. This demonstrates its superior adaptability across both homophilic and heterophilic graph structures. On citation graph datasets like Cora, Citeseer, and Pubmed, MbaGCN achieves competitive results, closely matching or surpassing the top-performing models. For example, on Cora, MbaGCN achieves an accuracy of 87.79%, just marginally lower than GCNII. This shows its effectiveness in handling strongly homophilic graphs and datasets with more complex structures. The model's strong performance can be attributed to its ability to flexibly aggregate neighborhood information, which allows it to capture local and global features while avoiding over-smoothing effectively, a common issue in deeper models.

▷ **Superior Performance on Heterophilic Datasets:** MbaGCN excels on heterophilic datasets, where traditional

Datasets	Cora	Citeseer	Pubmed	Computers	Photo	Actor	Wisconsin	Avg Rank
GCN	87.04±0.70 (2)	76.24±1.07 (2)	86.97±0.37 (2)	81.62±0.19 (2)	90.03±0.26 (2)	28.44±0.79 (2)	53.75±3.25 (2)	7.86
GAT	87.65±0.24 (2)	76.20±0.27 (2)	87.39±0.11 (2)	82.76±0.75 (2)	90.25±0.92 (2)	29.92±0.23 (2)	55.49±3.14 (2)	6.57
SGC	86.96±0.87 (2)	75.82±1.06 (2)	87.36±0.29 (2)	84.13±0.87 (2)	92.34±0.38 (2)	26.73±1.04 (2)	50.39±2.94 (2)	8.00
APPNP	87.71±0.76 (4)	76.66±1.22 (2)	87.76±0.43 (10)	84.51±0.34 (4)	88.97±0.96 (6)	29.68±0.72 (2)	59.80±1.96 (2)	5.71
GCNII	88.07±0.93 (64)	77.99±1.01 (64)	90.15±0.31 (64)	84.71±0.40 (8)	92.46±0.70 (4)	37.31±0.55 (8)	80.19±6.29 (10)	2.86
GPRGNN	87.75±0.62 (6)	77.08±1.06 (4)	89.36±0.33 (6)	87.43±0.49 (8)	94.36±0.31 (10)	33.87±0.58 (8)	81.02±2.94 (4)	2.71
SSGC	87.40±0.87 (6)	75.80±1.03 (2)	87.67±0.38 (4)	85.95±0.78 (4)	93.39±0.33 (4)	29.15±0.69 (2)	52.75±1.76 (2)	6.43
GGCN	87.73±1.24 (4)	76.63±1.49 (8)	89.08±0.47 (5)	90.36±0.52 (2)	94.23±0.65 (6)	37.54±1.46 (8)	85.88±4.19 (4)	3.14
MbaGCN (ours)	87.79±0.60 (10)	76.68±0.96 (6)	89.32±0.24 (8)	90.39±0.21 (4)	94.41±0.75 (2)	37.97±0.91 (10)	86.27±2.16 (8)	1.71

Table 1: Summary of classification accuracy (%) results. The best result for each benchmark is highlighted with a gray background, and the second-best result is emphasized with an underline. The layer configurations that achieve the best performance are recorded in brackets.

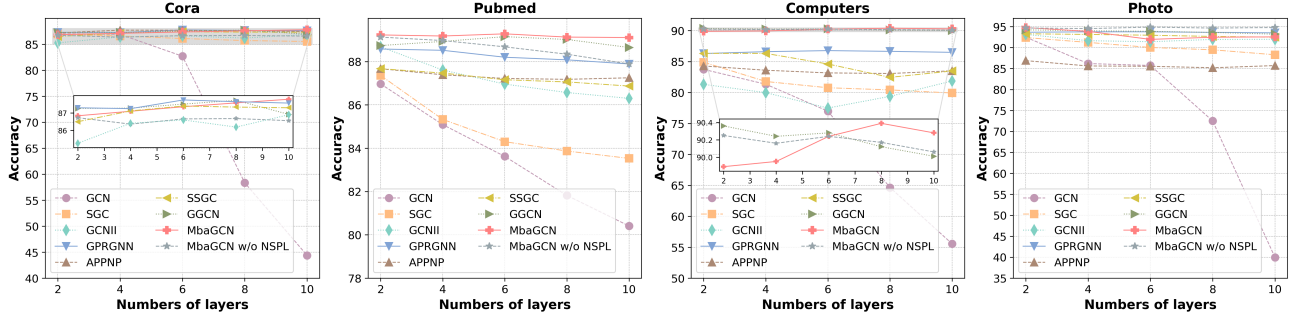


Figure 3: Performance of baselines and the proposed MbaGCN with 2/4/6/8/10 layers.

GNNs struggle due to adjacent nodes having dissimilar features. Standard GNNs often aggregate neighborhood information indiscriminately, leading to ineffective learning. In contrast, MbaGCN utilizes an adaptive aggregation mechanism inspired by Mamba, which adapts the aggregation process based on the relevance of the information. This allows MbaGCN to retain meaningful features and discard irrelevant ones, especially in heterophilic graphs. On the Wisconsin dataset, a typical heterophilic graph, MbaGCN achieves an accuracy of 86.27%, outperforming all other models. Similarly, on the Actor dataset, MbaGCN achieves 37.97%, demonstrating its robustness in heterophilic scenarios where traditional GNNs tend to underperform.

▷ **Consistency and Robustness:** In addition to excelling in heterophilic settings, MbaGCN maintains high performance across a wide variety of graph structures. While models like GCNII, GPRGNN, and GGCN perform well on specific datasets (e.g., GCNII on homophilic graphs), their overall rankings are lower compared to MbaGCN, highlighting the latter’s more consistent and robust performance across multiple types of datasets. This suggests that the flexibility and adaptability of the selective aggregation approach in MbaGCN allow it to handle a broad range of graph complexities and maintain high accuracy.

5.3 Impact of Layer Depth on Performance

Q: How does MbaGCN perform under different layer depths compared to baseline models? MbaGCN consistently maintains high performance and stability across vary-

ing numbers of layers, demonstrating its robustness in deeper architectures and its ability to mitigate over-smoothing.

▷ **Performance Trends Across Layer Depths:** Tab.2 summarizes the classification accuracy (%) of various GNN models across 2, 4, 6, 8, and 10 layers on the Actor and Wisconsin datasets. The results reveal a clear trend: GCN experiences significant performance degradation beyond 2 layers, highlighting its susceptibility to over-smoothing. Other deep GNN methods (e.g., GCNII) show better resilience, but their performance often peaks with shallow architectures, declining slightly at deeper depths. For example, APPNP on Actor and GCNII on Cora demonstrate marginal declines after 4 layers. The experimental results for the other two datasets (Cora and Citeseer) can be found in Appendix A.3.

▷ **MbaGCN’s Stability at Greater Depths:** In contrast, MbaGCN maintains consistent and competitive performance across all tested layer depths. Its selective state space mechanism enables it to prioritize important features and avoid the indiscriminate propagation of redundant information. For instance, on the Wisconsin dataset, MbaGCN achieves top accuracy even at 10 layers, significantly outperforming other models that degrade at similar depths. On homophilic datasets like Cora and Pubmed (see Fig. 3), MbaGCN achieves accuracy levels comparable to or exceeding the best-performing models, demonstrating its adaptability to both homophilic and heterophilic graph structures.

Layers	2	4	6	8	10	2	4	6	8	10
Dataset	Actor					Wisconsin				
GCN	28.44±0.79	27.18±0.51	26.93±0.55	26.56±0.38	26.55±0.43	53.75±3.25	51.96±2.95	48.63±3.53	48.04±4.12	47.00±4.51
SGC	26.73±1.04	24.98±0.47	24.97±0.71	25.09±0.76	25.19±0.79	50.39±2.94	50.00±4.12	50.20±3.33	49.41±3.53	48.82±3.92
APNP	29.68±0.72	28.77±0.75	28.38±0.71	28.55±0.82	28.35±0.66	59.80±1.96	59.22±2.36	58.63±2.35	59.41±3.14	57.84±2.75
GCNII	36.31±0.55	36.37±0.76	37.12±0.57	37.31±0.60	36.91±0.51	79.25±2.75	79.67±3.14	79.75±2.55	79.85±2.75	80.19±2.75
GPRGNN	32.62±0.66	32.62±0.97	33.34±0.60	33.87±0.58	33.60±0.58	79.08±3.92	81.02±2.94	78.12±2.95	76.67±2.16	75.10±2.94
SSGC	29.15±0.69	28.51±0.79	28.55±0.72	28.56±0.83	28.64±1.00	52.75±1.76	52.75±2.94	49.61±2.55	50.20±3.33	52.75±2.75
GGCN	37.22±1.29	37.46±1.16	37.50±1.42	37.54±1.46	37.25±1.28	84.51±4.06	85.88±4.19	84.12±4.51	84.31±4.38	84.12±4.76
MbaGCN (ours)	37.47±0.76	37.10±0.70	37.42±0.72	37.65±0.72	37.97±0.91	85.29±2.35	85.88±1.57	85.49±1.96	86.27±2.16	85.49±3.14
MbaGCN w/o NSPL	37.43±0.61	37.32±0.83	37.33±0.92	37.41±0.48	37.31±0.76	85.31±3.37	85.28±2.77	85.32±2.39	85.27±1.68	85.30±2.84

Table 2: Classification accuracy (%) comparison under different layer configurations. The best result under the same layer configuration is highlighted with a gray background, and the second-best result is emphasized with an underline.

Datasets	Citeseer	Actor	Wisconsin
Best	6	10	8
MbaGCN	76.68±0.96	37.97±0.91	86.27±2.16
MbaGCN w/o HL	75.87±0.73	35.39±2.49	82.55±2.35
Decline	1.06%	6.79%	4.31%
MbaGCN w/o IR	74.35±0.89	34.26±0.68	81.96±1.18
Decline	3.04%	9.77%	5.00%

Table 3: Ablation experiments of HL (HiPPO-LegS), IR (Input-Related) in proposed selective state space transition layer (S3TL).

5.4 Ablation Study

Q: How do NSPL, HiPPO-LegS (HL), and Input-Related (IR) contribute to the performance of MbaGCN, particularly in deeper layers? These components collectively enhance MbaGCN’s adaptability, mitigate over-smoothing, and preserve feature distinctiveness in deeper layers.

▷ **Impact of NSPL on Performance Across Depths:** As shown in Fig.3 and Tab.2, NSPL significantly enhances MbaGCN’s ability to maintain performance in deeper architectures. Without NSPL, MbaGCN performs best at shallow depths (e.g., 2 layers) but experiences a sharp decline as the depth increases. This is due to the model’s inability to regulate message flow within same-order neighborhoods, resulting in excessive information propagation or dilution of higher-order features. In contrast, incorporating NSPL allows dynamic control of message flow, preserving critical features and improving feature aggregation. For example, on the Wisconsin dataset, MbaGCN with NSPL achieves peak performance at 8 layers, while the ablated version struggles. However, on dense datasets like Photo, NSPL’s impact diminishes, likely due to optimization difficulties in dense graph structures, highlighting a potential area for future improvement.

▷ **Ensuring Robust Feature Propagation with HiPPO-LegS (HL):** HL in Eq.5 plays a pivotal role in maintaining robust feature propagation across deeper architectures. By ensuring efficient state transitions, HL minimizes the risk of feature degradation that often arises in deep GNNs due to over-smoothing. As shown in Tab.3, removing HL results in significant performance declines, such as a 6.79% drop on the

Actor dataset and a 2.48% drop on Wisconsin. These declines highlight HL’s critical role in preserving distinct node representations while enabling effective information aggregation across layers. Furthermore, HL’s impact becomes increasingly pronounced as the network depth grows, showcasing its ability to adapt state transitions dynamically and mitigate the compounding effects of over-smoothing.

▷ **Dynamic Adaptability Through Input-Related (IR) Matrices:** IR in Eq.7 enhances the adaptability of MbaGCN by generating state matrices that dynamically reflect node- and neighborhood-specific characteristics. This flexibility allows MbaGCN to balance the influence of local and global information, ensuring that critical features are neither overshadowed by higher-order information nor lost in the aggregation process. As depicted in Tab.3, the absence of IR results in notable performance degradation, such as a 9.77% accuracy drop on Actor and a 4.12% drop on Wisconsin. Compared to HL, IR demonstrates an even greater influence on model performance, particularly in datasets with diverse graph structures. This highlights IR’s key role in adapting state transitions to each graph’s structure, ensuring consistent performance across different depths and complexities.

6 Conclusion

This paper introduces **MbaGCN**, a novel architecture that integrates the Mamba paradigm into GNNs, addressing key challenges such as the loss of node-specific features in deeper architectures. By alternating between Message Aggregation Layers (MAL) and Selective State Space Transition Layers (S3TL), and incorporating the Node State Prediction Layer (NSPL), MbaGCN enables adaptive aggregation and propagation of information. Experimental results demonstrate MbaGCN’s strong performance across diverse datasets, particularly on heterophilic graphs. Ablation studies highlight the importance of key components like HiPPO-LegS (HL) and Input-Related (IR) in improving model adaptability and mitigating over-smoothing. While promising, MbaGCN faces challenges in dense graphs, suggesting opportunities for future work to optimize its components and extend its applicability to dynamic and multi-modal graphs. This study establishes a foundation for adaptive and scalable GNN architectures inspired by the Mamba paradigm.

Acknowledgments

This work was supported by a grant from the National Natural Science Foundation of China under grants (No.62372211, 62272191), and the Science and Technology Development Program of Jilin Province (No.20250102216JC).

Contribution Statement

Xin He and Yili Wang designed the experiments. Xin He performed the experiments. Yili Wang and Wenqi Fan analyzed the data. All authors contributed to writing and reviewing the manuscript. Xin He and Yili Wang contributed equally to this work. Xin Wang is the corresponding author.

References

- [Behrouz and Hashemi, 2024] Ali Behrouz and Farnoosh Hashemi. Graph mamba: Towards learning on graphs with state space models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 119–130, 2024.
- [Chen et al., 2020] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International conference on machine learning*, pages 1725–1735. PMLR, 2020.
- [Chien et al., 2020] Eli Chien, Jianhao Peng, Pan Li, and Olga Milenkovic. Adaptive universal generalized pagerank graph neural network. *arXiv preprint arXiv:2006.07988*, 2020.
- [Dao and Gu, 2024] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- [Ding et al., 2024] Yuhui Ding, Antonio Orvieto, Bobby He, and Thomas Hofmann. Recurrent distance filtering for graph representation learning. In *Forty-first International Conference on Machine Learning*, 2024.
- [Gasteiger et al., 2018] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997*, 2018.
- [Grazzi et al., 2024] Riccardo Grazi, Julien Siems, Simon Schrod, Thomas Brox, and Frank Hutter. Is mamba capable of in-context learning? *arXiv preprint arXiv:2402.03170*, 2024.
- [Gu and Dao, 2023] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [Gu et al., 2020] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33:1474–1487, 2020.
- [Gu et al., 2021] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.
- [He et al., 2021] Mingguo He, Zhewei Wei, Hongteng Xu, et al. Bernnet: Learning arbitrary graph spectral filters via bernstein approximation. *Advances in Neural Information Processing Systems*, 34:14239–14251, 2021.
- [Hu et al., 2025] Vincent Tao Hu, Stefan Andreas Baumann, Ming Gui, Olga Grebenkova, Pingchuan Ma, Johannes Fischer, and Björn Ommer. Zigma: A dit-style zigzag mamba diffusion model. In *European Conference on Computer Vision*, pages 148–166. Springer, 2025.
- [Jin et al., 2021] Wei Jin, Tyler Derr, Yiqi Wang, Yao Ma, Zitao Liu, and Jiliang Tang. Node similarity preserving graph convolutional networks. In *Proceedings of the 14th ACM international conference on web search and data mining*, pages 148–156, 2021.
- [Kipf and Welling, 2016] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [Li et al., 2018] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [Li et al., 2021] Ruifan Li, Hao Chen, Fangxiang Feng, Zhanyu Ma, Xiaojie Wang, and Eduard Hovy. Dual graph convolutional networks for aspect-based sentiment analysis. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6319–6329, 2021.
- [Lieber et al., 2024] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.
- [Miao et al., 2024] Rui Miao, Kaixiong Zhou, Yili Wang, Ninghao Liu, Ying Wang, and Xin Wang. Rethinking independent cross-entropy loss for graph-structured data. In *Proceedings of the 41st International Conference on Machine Learning*, pages 35570–35589, 2024.
- [Qin et al., 2024] Yifang Qin, Wei Ju, Hongjun Wu, Xiao Luo, and Ming Zhang. Learning graph ode for continuous-time sequential recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [Rusch et al., 2023] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. A survey on over-smoothing in graph neural networks. *arXiv preprint arXiv:2303.10993*, 2023.
- [Shen et al., 2024a] Xu Shen, Pietro Lio, Lintao Yang, Ru Yuan, Yuyang Zhang, and Chengbin Peng. Graph rewiring and preprocessing for graph neural networks based on effective resistance. *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [Shen et al., 2024b] Xu Shen, Yili Wang, Kaixiong Zhou, Shirui Pan, and Xin Wang. Optimizing ood detection in molecular graphs: A novel approach with diffusion models. In *Proceedings of the 30th ACM SIGKDD Conference*

- on *Knowledge Discovery and Data Mining*, pages 2640–2650, 2024.
- [Shen *et al.*, 2025] Xu Shen, Yixin Liu, Yili Wang, Rui Miao, Yiwei Dai, Shirui Pan, and Xin Wang. Raising the bar in graph ood generalization: Invariant learning beyond explicit environment modeling. *arXiv preprint arXiv:2502.10706*, 2025.
- [Tang *et al.*, 2023] Siyi Tang, Jared A Dunnmon, Qu Liangqiong, Khaled K Saab, Tina Baykaner, Christopher Lee-Messer, and Daniel L Rubin. Modeling multivariate biosignals with graph neural networks and structured state space models. In *Conference on Health, Inference, and Learning*, pages 50–71. PMLR, 2023.
- [Veličković *et al.*, 2017] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [Wan *et al.*, 2021] Sheng Wan, Shirui Pan, Jian Yang, and Chen Gong. Contrastive and generative graph convolutional networks for graph-based semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 10049–10057, 2021.
- [Wang *et al.*, 2021] Bo Wang, Tao Shen, Guodong Long, Tianyi Zhou, Ying Wang, and Yi Chang. Structure-augmented text representation learning for efficient knowledge graph completion. In *Proceedings of the Web Conference 2021*, pages 1737–1748, 2021.
- [Wang *et al.*, 2022] Yili Wang, Kaixiong Zhou, Rui Miao, Ninghao Liu, and Xin Wang. Adagcl: Adaptive subgraph contrastive learning to generalize large-scale graph training. In *Proceedings of the 31st ACM international conference on information & knowledge management*, pages 2046–2055, 2022.
- [Wang *et al.*, 2024a] Chloe Wang, Oleksii Tsepa, Jun Ma, and Bo Wang. Graph-mamba: Towards long-range graph sequence modeling with selective state spaces. *arXiv preprint arXiv:2402.00789*, 2024.
- [Wang *et al.*, 2024b] Yili Wang, Yixin Liu, Xu Shen, Chenyu Li, Kaize Ding, Rui Miao, Ying Wang, Shirui Pan, and Xin Wang. Unifying unsupervised graph-level anomaly detection and out-of-distribution detection: A benchmark. *arXiv preprint arXiv:2406.15523*, 2024.
- [Wang *et al.*, 2024c] Yili Wang, Kaixiong Zhou, Ninghao Liu, Ying Wang, and Xin Wang. Efficient sharpness-aware minimization for molecular graph transformer models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [Wang *et al.*, 2025] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Xiaocui Yang, Han Zhao, Daling Wang, and Yifei Zhang. Is mamba effective for time series forecasting? *Neurocomputing*, 619:129178, 2025.
- [Wu *et al.*, 2019] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *International conference on machine learning*, pages 6861–6871. PMLR, 2019.
- [Wu *et al.*, 2024] Xinyi Wu, Amir Ajorlou, Zihui Wu, and Ali Jadbabaie. Demystifying oversmoothing in attention-based graph neural networks. *Advances in Neural Information Processing Systems*, 36, 2024.
- [Xu *et al.*, 2021] Minghao Xu, Hang Wang, Bingbing Ni, Hongyu Guo, and Jian Tang. Self-supervised graph-level representation learning with local and global structure. In *International Conference on Machine Learning*, pages 11548–11558. PMLR, 2021.
- [Yan *et al.*, 2022] Yujun Yan, Milad Hashemi, Kevin Swersky, Yaoqing Yang, and Danai Koutra. Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. In *2022 IEEE International Conference on Data Mining (ICDM)*, pages 1287–1292. IEEE, 2022.
- [Yang *et al.*, 2024] Chenhongyi Yang, Zehui Chen, Miguel Espinosa, Linus Ericsson, Zhenyu Wang, Jiaming Liu, and Elliot J Crowley. Plainmamba: Improving non-hierarchical mamba in visual recognition. *arXiv preprint arXiv:2403.17695*, 2024.
- [Yu *et al.*, 2022] Pengyang Yu, Chaofan Fu, Yanwei Yu, Chao Huang, Zhongying Zhao, and Junyu Dong. Multiplex heterogeneous graph convolutional network. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2377–2387, 2022.
- [Yuan *et al.*, 2024] Xiaosong Yuan, Chen Shen, Shaotian Yan, Xiaofeng Zhang, Liang Xie, Wenxiao Wang, Renchu Guan, Ying Wang, and Jieping Ye. Instance-adaptive zero-shot chain-of-thought prompting. *arXiv preprint arXiv:2409.20441*, 2024.
- [Zhai *et al.*, 2024] Jiayu Zhai, Lequan Lin, Dai Shi, and Junbin Gao. Bregman graph neural network. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6250–6254. IEEE, 2024.
- [Zhang and Li, 2021] Muhan Zhang and Pan Li. Nested graph neural networks. *Advances in Neural Information Processing Systems*, 34:15734–15747, 2021.
- [Zhang *et al.*, 2021] Wentao Zhang, Mingyu Yang, Zeang Sheng, Yang Li, Wen Ouyang, Yangyu Tao, Zhi Yang, and Bin Cui. Node dependent local smoothing for scalable graph learning. *Advances in Neural Information Processing Systems*, 34:20321–20332, 2021.
- [Zhu and Koniusz, 2021] Hao Zhu and Piotr Koniusz. Simple spectral graph convolution. In *International conference on learning representations*, 2021.
- [Zhu *et al.*, 2024] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. *arXiv preprint arXiv:2401.09417*, 2024.

A Experiments Detail

Datasets	Nodes	Edges	Features	Classes
Cora	2,708	10,556	1,433	7
Citeseer	3,327	9,104	3,703	6
Pubmed	19,717	88,648	500	3
Computers	23,752	491,722	767	10
Photo	7,650	238,162	745	8
Actor	7,600	30,019	932	5
Wisconsin	251	515	1,703	5

Table 4: Dataset statistics.

A.1 Dataset

The statistics are listed in Tab. 4. Cora, Citeseer, Pubmed are citation graph datasets, Computers and Photo are web graph datasets, Actor and Wisconsin are heterogeneous graph datasets. We utilize the feature vectors, class labels, and 10 random splits as proposed by [Chen *et al.*, 2020] for citation graph datasets and heterogeneous graph datasets. We use the feature vectors, class labels, and 10 random splits as detailed in [He *et al.*, 2021] for web graph datasets.

- **Cora & Citeseer & Pubmed:** These three datasets are benchmark citation network datasets, where nodes represent papers and edges denote citation relationships.
- **Computers & Photo:** These two datasets are commonly used node classification datasets from the Amazon co-purchase graph. In these datasets, nodes represent goods, and edges indicate that two goods are frequently bought together.
- **Actor:** This is an actor-only subgraph of the film-director-actor-writer network, where nodes represent actors, and edges denote co-occurrence on the same Wikipedia page. Node features are derived from keywords in the Wikipedia pages.
- **Wisconsin:** This dataset consists of academic papers or webpages, with each paper or webpage treated as a node. Citation or hyperlink relationships between them are represented as edges.

A.2 Baselines

- **GCN:** This method is the most classic graph neural network, which enhances the model’s stability and scalability through the first-order approximation.
- **GAT:** This method is a graph neural network that uses an attention mechanism to explore node attributes across the graph, allowing for implicit weighting of different nodes within a neighborhood.
- **SGC:** This method is a simplified variant of GCN that removes nonlinearities and learnable parameter matrices between graph convolution layers.

- **APPNP:** This method improves the propagation scheme of GCN by leveraging personalized PageRank to enhance the performance of GCN models.
- **GCNII:** This method is a variant of GCN that incorporates residual connections and identity mapping, effectively alleviating the over-smoothing phenomenon.
- **GPRGNN:** This method is a GNN model that adapts the PageRank algorithm within a GNN to capture node importance, and improve performance on tasks like classification and link prediction, while also alleviating the over-smoothing issue.
- **SSGC:** This method mitigates over-smoothing by using a modified Markov diffusion kernel, which symmetrically scales the aggregation process to preserve node feature distinctiveness in deeper layers.
- **GGCN:** This method solves over-smoothing in both homophilic and heterophilic graphs through structure-based and feature-based edge correction.

A.3 Impact of Layer Depth on Performance

Tab. 5 summarizes the classification accuracy (%) of various GNN models across 2, 4, 6, 8, and 10 layers on the Cora and Citeseer datasets. GCN suffers from significant performance degradation beyond 2 layers due to its susceptibility to over-smoothing. In contrast, deeper GNN methods, such as GCNII, exhibit greater robustness, with their performance remaining stable or even improving steadily as the number of layers increases.

From the performance changes observed in our proposed MbaGCN and its ablation version MbaGCN w/o NSPL as the number of layers increases, the introduction of NSPL enhances the model’s ability to mitigate the over-smoothing problem. However, it also reduces the stability of the model’s performance. For instance, on the Citeseer dataset, the inclusion of NSPL results in greater fluctuations in MbaGCN’s performance as the number of layers increases, which may be attributed to the increased optimization difficulty of NSPL in deeper architectures.

A.4 Time And Memory Cost

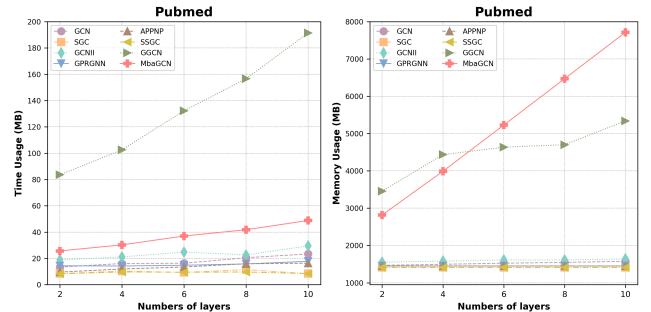


Figure 4: Performance of baselines and the proposed MbaGCN with 2/4/6/8/10 layers.

To further investigate the efficiency and scalability of our model, we conduct experiments to analyze the trends in time

Layers	2	4	6	8	10	2	4	6	8	10
Dataset	Cora					Citeseer				
GCN	87.04±0.70	86.90±0.78	82.70±0.78	58.35±5.05	44.39±2.48	76.24±1.07	75.59±1.00	66.86±2.66	41.17±4.50	37.42±3.15
SGC	86.96±0.87	86.58±0.74	86.10±0.60	85.71±0.74	85.53±0.78	75.82±1.06	75.15±0.72	74.46±0.90	74.04±0.90	73.87±0.89
APPNP	87.18±0.89	87.71±0.76	<u>87.69±0.91</u>	87.59±0.83	87.44±0.91	<u>76.66±1.22</u>	75.64±1.04	75.96±0.95	75.97±0.92	75.80±1.09
GCNII	85.26±0.76	86.41±0.91	86.60±1.39	86.19±1.21	86.90±0.87	74.72±1.24	74.48±1.13	74.94±1.36	75.63±1.18	76.01±1.26
GPRGNN	87.30±0.87	<u>87.25±0.79</u>	87.75±0.62	<u>87.65±0.91</u>	<u>87.57±0.82</u>	76.89±1.28	77.08±1.06	76.96±1.01	<u>76.54±1.00</u>	<u>76.57±0.91</u>
SSGC	86.50±0.72	87.12±0.87	87.40±0.87	87.34±0.78	87.30±0.87	75.80±1.03	75.69±1.23	75.55±0.94	75.57±0.88	75.21±0.76
GGCN	<u>87.26±1.27</u>	87.28±1.28	87.51±1.42	87.73±1.24	86.92±0.99	76.63±1.54	<u>76.53±1.43</u>	76.47±1.80	76.63±1.49	76.60±1.60
MbaGC (ours)	86.84±0.85	87.10±0.80	87.36±0.76	87.58±1.29	<u>87.79±0.60</u>	76.03±0.91	75.91±1.20	<u>76.68±0.96</u>	76.03±0.96	75.80±1.40
MbaGCN w/o NSPL	86.73±0.75	86.36±0.91	86.66±0.86	86.68±1.18	86.56±0.74	76.23±0.94	76.28±1.49	76.16±0.88	76.19±1.41	76.12±1.22

Table 5: Classification accuracy (%) comparison under different layer configurations. The best result under the same layer configuration is highlighted with a gray background, and the second-best result is emphasized with an underline.

and memory usage as the number of layers increases. The results are then compared with all baseline methods. The result of the experiment is shown in Fig. 4.

From the figure, we can observe that as the number of layers increases, most baseline methods exhibit no significant growth in time and memory consumption. This is because these methods primarily increase the number of layers by adding simple aggregation operations without introducing additional learnable parameters, resulting in a relatively flat growth in both time and space consumption. In contrast, our proposed MbaGCN exhibits a gradual increase in time consumption and a linear increase in memory consumption. This is due to the fact that the number of parameters in MbaGCN is proportional to the number of layers, meaning that as the number of layers increases, the model’s parameter count increases accordingly, leading to a linear rise in memory usage.

Although our model shows differences from the baseline methods in terms of time and memory consumption, it presents a novel approach by solving the over-smoothing problem via the selection state space model and improving the model’s expressive capacity. Future work will focus on optimizing the algorithm to reduce time and memory consumption, further enhancing the model’s potential for large-scale graph data applications.